Act as an expert researcher. Explore this repos throughly. * My new research component tcwpn: https://github.com/dulhara79/tcwpn_test * Main research repo: https://github.com/dulhara79/R26-DS-012 * Doctor-side mobile app: https://github.com/dulhara79/tcwpn_mobile_app * Patientside mobile app: has files section (DewduSendanayake / anxiety_mobile_app) I want to know how to connect these section together and integrate this mobile app. we innitially decided to get huggingface spaces, model hosting and connect thoese together. * digital phenotyping from patients mobile phone app-> member 1 huggingface host model -> send output to fusion model hosted in huggingface * physiological signals from patients mobile phone app -> member 2 huggingface host model -> send output to fusion model hosted in huggingface * contextual data from patients mobile phone app -> member 3 huggingface host model -> send output to fusion model hosted in huggingface * each patients clinical notes from doctors mobile phone app -> member 4 huggingface host model -> send output to fusion model hosted in huggingface * fusion model gives final risk score and provide it to the patient

I've cloned and read all three repos plus the patient-app files in project knowledge. Before the architecture answer, there are findings that change what you can honestly wire together — so I'll give you the verified state first, then the integration design.

---

1. What is actually there right now

Piece	Status in code	Reality
TC-WPN inference service	**Does not exist**	tcwpn_test has zero serving code — no FastAPI, no Gradio, no Dockerfile, no app.py. Only scripts/train.py and scripts/evaluate.py. The doctor app's README points at dulharakaushalya-tc-wpn-demo.hf.space; I could not verify that Space exists, and no code in any repo

Piece	Status in code	Reality
		builds it.
Fusion service	**Does not exist**	tcwpn_mobile_app/lib/data/api/gateways.dart calls POST /contribute and GET /state/{mrn}. No repo contains a server implementing those routes. Env.fusionBase is an empty String.fromEnvironment, so Env.hasFusion is false and every fusion call currently falls into the FusionResult.local(...) branch.
C1 physiological	**Live**	HF-Anxiety/main.py, FastAPI, deployed at dewdu-physiological-anxiety-escalation.hf.space. Real /ingest, /predict/{user_id}, /history, /feedback.
C2 behavioural	**Withdrawn by its own author**	The patient app's digital_phenotyping_page.dart header documents why the risk score was removed: GLOBEM AUROC 0.548 [0.516–0.588] vs permutation null 0.487, cross-cohort transfer 0.478, net benefit ~0.6 cases per 1,000. The page now emits descriptive observations only.
C3 intervention	**Live-ish**	Real FastAPI at c3_api/, artifacts committed

Piece	Status in code	Reality
		(XGBoost + calibrator + conformal + FAISS).

Security, fix today: C3_Personalised_Anxiety_Intervention_Framework/.env is committed to a public repo with a real C3_SECRET_KEY (the JWT signing key). Rotate it, add .env to .gitignore, and purge it from history.

---

2. Three things that block the architecture as you described it

(a) There is no fusion model to host — C3 is not one

c3_api/artifacts/feature_cols.json records:

```
"composite_risk_formula": "(0.25*physiological_risk + 0.20*behavioral_risk + 0.40*textual_risk) / 0.85"
"target_col": "risk_tier"
```

composite_risk is a *deterministic linear function* of the three modality inputs, it is fed to the model as feature F9, and the target risk_tier is a tiering of that same quantity. The model is largely learning to invert a formula you already wrote down. The SHAP table in the same file confirms the distortion: physiological_risk 2.832, composite_risk 1.085, textual_risk **0.079**. Your TC-WPN output has almost no influence on the tier it produces.

Also: n_nhanes: 2705, n_colombia: 2657 — it was trained on survey data (NHANES GAD-7 imputed via ordinal regression, Colombian PSS-10/PHQ-9), not on any patient who ever wore your chest strap or had a note read by TC-WPN.

Calling C3's output "the fused multimodal risk score" is the single easiest thing for a reviewer or your external supervisor to take apart.

(b) The weights disagree in three places

Source	C1 physio	C2 behav	C3 interv	C4 text
R26-DS-012/ README.md	0.40	0.35	—	0.25
c3_api/ artifacts/ feature_cols.jso n	0.25	0.20	—	0.40

Source	C1 physio	C2 behav	C3 interv	C4 text
tcwpn_mobile_app/lib/core/config/env.example.dart	0.25	0.20	0.15	0.40

Nobody can reproduce a composite score until one of these is the definition and the other two are deleted.

(c) TC-WPN's own mechanisms are not supported by your evidence

This is your data, from data/TC-WPN — Phase 4.../phase4_paired_deltas.csv, 5 seeds, paired:

config	mean AUROC	Δ vs aux_only	paired t p	seeds better	Cohen's dz
aux_only	0.7371	—	—	—	—
temporal_aux	0.7377	+0.0006	0.906	2/5	0.056
pcw_aux	0.7291	−0.0080	0.150	1/5	−0.797
tcwpn_full	0.7377	+0.0006	0.886	1/5	0.068

Neither w^T nor w^C produces a detectable improvement once the auxiliary CE head is held constant. The Stage C DeLong "TC-WPN vs FAIR baseline, ΔAUROC 0.239, p=2.1e-52" is measuring the auxiliary head — every aux-free arm had proto_cos ≥ 0.9997 and p_sd ≤ 0.0018, i.e. representation collapse, not a baseline.

Consequence for integration: the thing you deploy is ProtoNet + learnable τ + auxiliary CE head, AUROC ≈ 0.738 (0.718–0.758) on a *concurrent-detection* MIMIC-IV task at prevalence 0.594. It carries the name TC-WPN, but the endpoint contract, the app copy, and the paper must not claim the temporal or consistency mechanism is what produces the score.

Two more things the doctor app assumes that don't exist: TcwpnResult.calibratedProbability and .confidence. TC-WPN outputs a softmax over a cosine head with ECE ≈ 0.085 and Brier ≈ 0.209 — there is no calibrator in the repo. Weighted fusion of an uncalibrated score is arithmetic, not probability.

---

3. Concrete client-side bugs, if you wire it as-is

C3Gateway.classify() in gateways.dart will return **422** against the deployed C3 API every time:

Bug	Detail
Missing required field	FeatureVector.composite_risk is Field(...) (required). The gateway never sends it.
Nulls into non-optional floats	physiological_risk, behavioral_risk, textual_risk, last_reward_norm are float = Field(..., ge=0, le=1). The gateway's doc comment says it passes null "so the service can impute" — the service cannot; it rejects.
Range violation	models.dart:128 maps unknown gender to 3; schema is ge=1.0, le=2.0.
Wrong risk shape	RiskLevel/3-tier from C3 vs TC-WPN's continuous probability — the fusion_detail_screen treats them as interchangeable modality scores.

Also: README.md in tcwpn_mobile_app describes the old two-module launcher (shell/portal_launcher.dart, lib/tcwpn/, lib/c3/) — none of which exist in lib/ any more. And R26-DS-012/README.md still documents TC-WPN with the **archived** entropy formulation w_confidence = 1/(1+β·H(x)), which src/tcwpn/model.py replaced with w^C = exp(β·max(cos(z_i, p̃_c), 0)). gateways.dart repeats the archived exp(−λΔt/365), λ=0.5 as a fixed constant; in the current model λ is a *learned* log_lambda.

---

4. The integration I'd recommend

Keep your topology, but change what the fusion node *is*.

Do not build a fusion model. Build a fusion registry.

```
Patient app ──/contribute─┐
Doctor app  ──/contribute─┤   Fusion Gateway (single FastAPI Space)
C1 Space    ──/contribute─┤   • stores latest reading per modality per patient
                          │   • stores captured_at, model_version, status
```

```
                └─→ • serves GET /state/{pid} — a panel, not a number
                    • composite ONLY under stated preconditions
```

Why a registry and not a model: you have **zero patients with all four modalities and a ground-truth label**. You cannot fit or validate fusion weights. A registry is defensible today; a fusion model is not, and would need its own IRB-covered cohort.

One contract, five fields

```
POST /contribute
{
  "patient_id": "pseudonymous-id",
  "component":  "c1_physiological" | "c2_behavioral" | "c3_intervention" |
"c4_clinical_nlp",
  "score":      0.0-1.0,
  "captured_at": "ISO-8601 UTC",
  "model_version": "tcwpn-aux-k5-seed42",
  "status": "ok" | "insufficient_data" | "not_validated",
  "score_semantics": "p_anxiety_uncalibrated"
}
```

status and score_semantics are the fields that make this honest. C2 posts status: "not_validated" — its score is displayed and never enters any composite. C1 posts insufficient_data until calibration completes. The gateway never silently renormalises over missing modalities; GET /state returns which modalities reported and which didn't, and the UI says so.

Ordered build sequence

1. **Rotate the leaked C3_SECRET_KEY.** Nothing else until this is done.
2. **Write TC-WPN's serving Space.** It doesn't exist. Minimum: POST /predict taking {note_text, note_date, support_set[], visit_count}, loading the aux_only-or-tcwpn_full checkpoint, running build_prototype + classify from src/tcwpn/model.py forward-only. Decide and document where the support set comes from — MIMIC-IV episodes or NHSL notes. These are different claims, and only the second is few-shot clinical adaptation.
3. **Fix the four C3Gateway schema violations** or fix the C3 schema to accept Optional[float] and impute server-side. Pick one; document which.
4. **Pick one weight set** and delete the other two. Given (a), I'd argue for no fixed weights at all in v1.
5. **Build the fusion registry Space.** ~200 lines of FastAPI + SQLite/Postgres. It has no ML in it, and that is the point.

6. **Point Env.fusionBase, Env.tcwpnBase, Env.c3Base** at the three Spaces and delete Env.defaultWeights and the local-composite fallback path.

Operational caveats for HF Spaces

Free-tier Spaces sleep and lose disk on restart — a registry needs external persistence (Supabase/Neon/Postgres), not local files. C1's set_norm_params already holds per-user calibration "in server memory", which is lost on every restart; the registry must not repeat that. Cold start regularly exceeds 60s, which is why Env.inferenceTimeout is 180s. And Spaces are public HTTP endpoints with no auth by default — do not put real NHSL note text or identifiable IDs through one without a token gate and a pseudonymous ID scheme.

---

5. What I could not verify

- Whether dulharakaushalya-tc-wpn-demo.hf.space exists or what it serves.
- What checkpoint you intend to deploy — no .pt/.ckpt file is committed to tcwpn_test.
- Whether risk_tier in C3's training data is a binning of composite_risk. The feature file strongly implies it, but the label-construction script isn't in the repo. If you can get that script from the C3 owner, it settles whether the circularity is real or only apparent.
- Whether the patient-side app has an anxiety_mobile_app fusion write path beyond the placeholder in api_service.dart (PLACEHOLDER_FUSION_ENDPOINT).

The most useful next artifact you could give me is the C3 label-construction notebook and whatever exists of the TC-WPN Space. With those I can write you the exact /predict handler and the registry service.

these are the repos we used to create our models: * My new research component tcwpn: https://github.com/dulhara79/tcwpn_test * Main research repo: https://github.com/dulhara79/R26-DS-012 these are the repos we used to create mobile app: * D ctor-side mobile app: https://github.com/dulhara79/tcwpn_mobile_app * Patientside mobile app: has files section (DewduSendanayake / anxiety_mobile_app) we still haven create hugging face spaces and models to host the models. so i want step by step guidance to how to do that? there have 5models: 1. physiological(dewdu compo) hosted in herHF 2. digital pheno(senu compo) hosted in herHF 3. contextual(uvindu compo) hosted in his HF 4. clinical text tcwpn(dulhara my

compo) hosted in my HF 5. fusion model(uvindu compo) hosted in his HF i want ot know howto integrte thoese? this should provide output by separating the patient wise. this is a industrial commercial app demo, since this is demo we used huggingface and free services to show the demo working and other operation stuff should in the very standard way

Before the runbook: **a premise in your plan no longer holds, and it changes the first decision.**

As of 2026, Gradio and Docker Spaces run on compute and require a paid plan to create: PRO for personal accounts, Team or Enterprise for organizations. Free personal accounts in good standing can still host up to 2 Gradio Spaces running on ZeroGPU. A FastAPI REST service needs a **Docker** Space, so "5 free HF Spaces" isn't available to you the way it was when this plan was made. Dewdu's existing Space (dewdu-physiological-anxiety-escalation.hf.space) predates the change and appears grandfathered — verify before assuming.

Also worth knowing before you design persistence: Each Spaces environment is limited to 16GB RAM, 2 CPU cores and 50GB of (not persistent) disk space by default. Nothing written to disk survives a restart, which rules out SQLite-on-Space for the patient registry.

So step 0 is a team decision, and it's the one thing I can't make for you:

Option	Cost	What you get
A — 4× PRO	$9/mo each, ~$36/mo	Docker Spaces on CPU Basic, each member owns their own, matches your plan exactly
B — HF model repos + Render/Cloud Run	Free	Model weights live in each member's free HF *model repo* (the markable artifact), serving containers run on Render free web services. Cold-start and sleep behaviour is comparable to what you expected from HF.

Option	Cost	What you get
C — Gradio on ZeroGPU	Free	2 per account, ZeroGPU daily quota ~5 min. Not viable for an always-on demo API.

I'd take **A** if the team can spend ~$110 over the demo period, **B** otherwise. Everything downstream in the runbook is identical either way — only the `Dockerfile` host changes.

I've written the full integration runbook as a file you can share with the other three:

R26-DS-012_integration_runbook.md

R26-DS-012 — Model Hosting & Integration Runbook

Scope: deploying 5 models as independent services and integrating them into the patient app and the clinician app, with per-patient separation.

Audience: all four component owners.

Status of this document: engineering plan. Section 10 lists the scientific constraints that must be respected in what the services *claim*, and those are not negotiable by the engineering.

---

0. Decisions that must be made before anyone writes code

#	Decision	Owner	Default if undecided
0.1	Hosting tier (PRO Docker Spaces vs Render)	Team	Blocks everything
0.2	Canonical patient identifier scheme (§1)	Team	Blocks everything
0.3	One database	Uvindu (fusion)	Blocks §4

#	Decision	Owner	Default if undecided
	instance + who owns the credentials		
0.4	Whether the demo uses synthetic patients or real consented participants	Team + Dr. Suraweera	Determines whether §8.6 applies

Do not start §5 until 0.1–0.3 are settled. Every service depends on them.

---

1. Patient-wise separation — solve this first

This is the part you asked about explicitly, and it is currently broken across the two apps.

1.1 The problem, as it exists in the code today

App	Identifier	Where it comes from
Patient app (anxiety_mobile_app)	user_id	SharedPreferences, set at enrolment, passed to /ingest, /predict/{user_id}
Clinician app (tcwpn_mobile_app)	patientMrn	Typed by the clinician, used as patient_id in TcwpnGateway.analyse() and as mrn in FusionGateway.state(mrn)

These are two different namespaces. If C1 writes under user_id = "P007" and C4 writes under mrn = "NHSL-2026-114", the fusion service holds two unrelated records and no patient is ever complete. **No amount of downstream work fixes this; it has to be fixed at the identity layer.**

1.2 The design

Introduce one canonical `subject_id` — a UUIDv4, generated once, meaningless on its own. Every service reads and writes only `subject_id`. The app-local identifiers become *aliases*.

```
subjects
  subject_id      uuid    PRIMARY KEY
  enrolled_at     timestamptz
  study_arm       text
  active          boolean

subject_aliases
  alias           text          -- "P007" or "NHSL-2026-114"
  alias_type      text          -- 'app_user_id' | 'clinical_mrn'
  subject_id      uuid REFERENCES subjects
  PRIMARY KEY (alias_type, alias)
```

The clinical MRN never leaves the hospital-side app in plaintext. Store `sha256(mrn + PEPPER)` as the alias, with `PEPPER` held as a server-side secret. The clinician app hashes before sending. That way a leak of the fusion database does not expose hospital record numbers.

1.3 The enrolment flow (pairing)

```
1. Clinician enrols patient in ClinAnx  →  POST /v1/subjects

                                              {alias_type:'clinical_mrn', alias: sha256(mrn)}
                                          ←  { subject_id, pairing_code: "4F2K-8Q1M" }
                                              (pairing_code, 6 chars, expires in 24h)

2. Clinician reads the 6-char code (or shows a QR) to the patient.

3. Patient app enrolment screen  →  POST /v1/subjects/pair

                                          { pairing_code, app_user_id }
                                      ←  { subject_id }

4. Patient app stores subject_id in SharedPreferences alongside user_id

   and sends subject_id on every subsequent call.
```

Pairing code, not a shared link — it is one screen in each app, it works offline in the clinic, and it makes consent an explicit act rather than a silent join.

1.4 Migration for existing app code

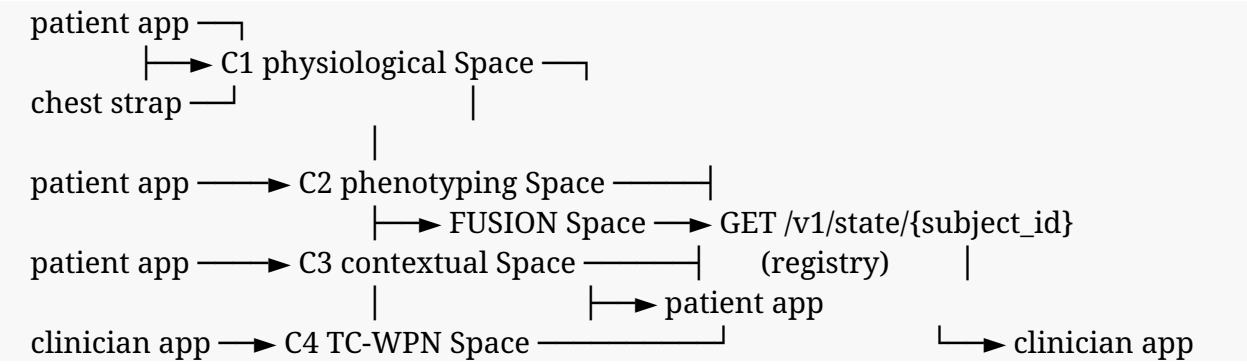
- **Patient app:** ParticipantIdentityService already has migrateLegacyIdentity(). Extend it to hold subject_id, and change ApiService.baseUrl calls so user_id stays for C1's own InfluxDB writes but subject_id is what goes to the fusion service.
- **Clinician app:** Patient.mrn stays for display. Add Patient.subjectId. FusionGateway.state(mrn) becomes state(subjectId).

---

2. Topology — push registry, not synchronous fan-out

Two designs were possible. Take the first.

2.1 Chosen: push-based contribution



Each modality service pushes its result to the fusion service whenever it computes one. The fusion service stores the latest reading per modality per subject and serves a read-only view.

2.2 Rejected: fusion calls the four models on demand

Rejected for four concrete reasons:

1. **Cold starts compound.** Free-tier Spaces sleep. A synchronous fan-out across four sleeping Spaces is four sequential cold starts. The clinician app already sets inferenceTimeout = 180s for one. Four would exceed any reasonable timeout.

2. **The cadences are incompatible.** C1 produces a value every 60 seconds,

   C2 daily, C4 only when a clinician writes a note. There is no single moment at which asking all four is meaningful.

3. **Failure coupling.** One sleeping Space would take down the whole read path.

4. **The clinician app already assumes push.** gateways.dart implements

   POST /contribute + GET /state/{mrn}. Build what the client expects.

---

3. The contract — identical in all five services

One file, copy-pasted byte-identical into every Space. Do not let it drift.

app/contract.py:

```
"""Shared inter-component contract. IDENTICAL COPY in all five services.
Any change here is a coordinated change across five repos. Bump CONTRACT_VERSION."""
from __future__ import annotations
from datetime import datetime
from enum import StrEnum
from typing import Any, Literal, Optional
from pydantic import BaseModel, Field

CONTRACT_VERSION = "1.0.0"

class Component(StrEnum):
    C1_PHYSIOLOGICAL = "c1_physiological"
    C2_BEHAVIORAL    = "c2_behavioral"
    C3_CONTEXTUAL    = "c3_contextual"
    C4_CLINICAL_NLP  = "c4_clinical_nlp"

class ReadingStatus(StrEnum):
    OK               = "ok"               # usable score
    INSUFFICIENT_DATA  = "insufficient_data"   # model ran, not enough input
    NOT_VALIDATED      = "not_validated"      # score exists, display only, never fused
    MODEL_ERROR        = "model_error"

class Contribution(BaseModel):
    subject_id:      str
```

```
    component:       Component
    score:           Optional[float] = Field(None, ge=0.0, le=1.0)
    score_semantics: str          # e.g. "p_anxiety_uncalibrated"
    status:          ReadingStatus
    captured_at:     datetime     # when the DATA was captured, not when posted
    model_version:   str          # "tcwpn-aux-k5-seed42@2026-08-14"
    detail:          dict[str, Any] = Field(default_factory=dict)
    contract_version: str = CONTRACT_VERSION

class ModalitySnapshot(BaseModel):
    component:       Component
    score:           Optional[float]
    score_semantics: str
    status:          ReadingStatus
    captured_at:     Optional[datetime]
    age_seconds:     Optional[int]
    stale:           bool         # age > 72h
    model_version:   Optional[str]

class SubjectState(BaseModel):
    subject_id:  str
    modalities:  list[ModalitySnapshot]
    reporting:   list[Component]     # status == ok AND not stale
    missing:     list[Component]
    composite:   Optional[float]     # None unless §7 preconditions met
    composite_basis: str          # human-readable, always populated
    generated_at: datetime
    contract_version: str = CONTRACT_VERSION
```

Two fields carry all the honesty in this system:

- status — not_validated means the score renders in the UI with its caveat

  and is *never* included in any composite. This is how C2 participates in the demo without making a claim its validation does not support.

- score_semantics — a free-text label saying what the number means. C1's is a

  10-minute escalation forecast peak; C4's is a note-level anxiety probability. These are not the same quantity and the string stops anyone from averaging them without noticing.

---

4. Persistence

Spaces disk is **not persistent**. State must live outside the container.

Use one managed Postgres. Supabase free tier or Neon free tier. One instance, owned by the fusion service, credentials in HF Space secrets.

```
-- run once, in the fusion Space's database

CREATE TABLE subjects (
  subject_id   uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  enrolled_at  timestamptz NOT NULL DEFAULT now(),
  study_arm    text,
  active       boolean NOT NULL DEFAULT true
);

CREATE TABLE subject_aliases (
  alias_type text NOT NULL,
  alias      text NOT NULL,
  subject_id uuid NOT NULL REFERENCES subjects(subject_id) ON DELETE CASCADE,
  PRIMARY KEY (alias_type, alias)
);

CREATE TABLE pairing_codes (
  code       text PRIMARY KEY,
  subject_id uuid NOT NULL REFERENCES subjects(subject_id) ON DELETE CASCADE,
  expires_at timestamptz NOT NULL,
  used_at    timestamptz
);

CREATE TABLE modality_readings (
  id           bigserial PRIMARY KEY,
  subject_id   uuid NOT NULL REFERENCES subjects(subject_id) ON DELETE
CASCADE,
  component    text NOT NULL,
  score        numeric,
  score_semantics text NOT NULL,
  status       text NOT NULL,
  captured_at  timestamptz NOT NULL,
  received_at  timestamptz NOT NULL DEFAULT now(),
  model_version  text NOT NULL,
  detail       jsonb NOT NULL DEFAULT '{}'::jsonb
);
```

```
-- append-only history; the "current" reading is the newest per (subject, component)
CREATE INDEX idx_readings_lookup
  ON modality_readings (subject_id, component, captured_at DESC);

CREATE TABLE audit_log (
  id         bigserial PRIMARY KEY,
  at         timestamptz NOT NULL DEFAULT now(),
  actor      text NOT NULL,      -- component name or 'clinician:<id>'
  action     text NOT NULL,
  subject_id uuid,
  detail     jsonb
);
```

Append-only readings, never UPDATE. The current state is a query, the history is free, and you can reconstruct exactly what the clinician saw at any past moment. That last property is what a clinical-decision-support audit requires, and it costs you nothing to have now.

Row-level isolation per patient is enforced by `subject_id` being on every query — the API layer must never accept a query without one, and never return a list of all subjects to a patient-app token.

---

5. The canonical Space scaffold

Every one of the five services uses this identical layout. Differences between services live only in `app/inference.py`.

5.1 Repository layout

```
<space-repo>/
├── README.md          # YAML front matter — REQUIRED, controls the Space
├── Dockerfile
├── requirements.txt
├── app/
│   ├── __init__.py
│   ├── main.py        # FastAPI app, routes, health
│   ├── contract.py    # §3 — byte-identical in all five
│   ├── settings.py    # env vars, no hardcoded secrets
│   ├── security.py    # bearer-token auth
│   └── inference.py   # THE ONLY FILE THAT DIFFERS PER COMPONENT
```

```
└── tests/
    └── test_contract.py
```

Do **not** commit model weights into the Space repo. See §5.5.

5.2 README.md front matter

```
---

title: TC-WPN Clinical NLP Service
emoji: 🏥
colorFrom: blue
colorTo: indigo
sdk: docker
app_port: 7860
pinned: false
license: other
short_description: Few-shot clinical anxiety detection (R26-DS-012 C4)

---
```

app_port: 7860 is mandatory. Docker Spaces are proxied to 7860 and nothing else.

5.3 Dockerfile

```
FROM python:3.11-slim

# Spaces run the container as a non-root user; HOME is not reliably writable.
RUN useradd -m -u 1000 appuser
WORKDIR /app

COPY --chown=appuser:appuser requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

COPY --chown=appuser:appuser . .

USER appuser
# transformers/huggingface_hub need a writable cache under a path we own
ENV HF_HOME=/app/.cache \
    TRANSFORMERS_CACHE=/app/.cache \
    PYTHONUNBUFFERED=1
```

```
EXPOSE 7860
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "7860", "--workers", "1"]
```

--workers 1 is deliberate: 2 CPU cores, and a second worker would load a second copy of Bio_ClinicalBERT into a 16 GB budget.

5.4 app/security.py

```
import hmac, os
from fastapi import Header, HTTPException, status

def _tokens(var: str) -> set[str]:
    return {t.strip() for t in os.getenv(var, "").split(",") if t.strip()}

SERVICE_TOKENS = _tokens("SERVICE_TOKENS")   # component-to-component
CLIENT_TOKENS  = _tokens("CLIENT_TOKENS")    # mobile apps

def _check(authorization: str | None, allowed: set[str]) -> str:
    if not allowed:
        raise HTTPException(500, "Service misconfigured: no tokens set")
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Missing bearer token")
    presented = authorization.removeprefix("Bearer ")
    for t in allowed:                        # constant-time, no early exit
        if hmac.compare_digest(presented, t):
            return t
    raise HTTPException(status.HTTP_403_FORBIDDEN, "Invalid token")

async def service_auth(authorization: str | None = Header(None)):
    return _check(authorization, SERVICE_TOKENS)

async def client_auth(authorization: str | None = Header(None)):
    return _check(authorization, CLIENT_TOKENS)
```

Two token classes, because a compromised phone must not be able to forge a C1 contribution.

5.5 Model weights: HF model repo, not the Space repo

Each member creates a **model repository** (free, unaffected by the Spaces pricing change) and pushes weights there:

```
pip install huggingface_hub
huggingface-cli login
```

```
huggingface-cli repo create tcwpn-clinical-anxiety --type model
git clone https://huggingface.co/<user>/tcwpn-clinical-anxiety
cd tcwpn-clinical-anxiety
git lfs install
cp /path/to/tcwpn_full_k5_seed42.pt .
cp /path/to/config.yaml .
# write a model card: training data, metrics, intended use, limitations
git add . && git commit -m "TC-WPN aux-head checkpoint, seed 42" && git push
```

The Space downloads at startup:

```
from huggingface_hub import hf_hub_download
ckpt = hf_hub_download("<user>/tcwpn-clinical-anxiety",
                       "tcwpn_full_k5_seed42.pt",
                       token=os.getenv("HF_TOKEN"))
```

Three reasons this is better than committing weights into the Space:

- The model repo is the artifact your examiners can cite and version.
- Space rebuilds stay fast; weights are cached.
- Private model repo + HF_TOKEN secret keeps a checkpoint out of public view

  while the Space itself stays reachable.

5.6 Secrets

Space → Settings → Variables and secrets → New secret. Never in git.

Secret	C1	C2	C3	C4	Fusion
FUSION_BASE_URL	✓	✓	✓	✓	—
FUSION_SERVICE_TOKEN	✓	✓	✓	✓	—
SERVICE_TOKENS (csv, inbound)	—	—	—	—	✓
CLIENT_TOKENS	✓	✓	✓	✓	✓

Secret	C1	C2	C3	C4	Fusion
(csv, inbound)					
DATABASE_URL	—	—	—	—	✓
MRN_PEPPER	—	—	—	—	✓
HF_TOKEN	✓	✓	✓	✓	—

Generate every token with `python -c "import secrets; print(secrets.token_urlsafe(32))"`.

> **Do now, before anything else:**
> `C3_Personalised_Anxiety_Intervention_Framework/.env` is committed to the public `R26-DS-012` repo with a live `C3_SECRET_KEY`. Rotate it, gitignore the file, and purge it from history with `git filter-repo`.

---

6. The five services, one at a time

6.1 C1 — Physiological (Dewdu)

Already deployed. `dewdu-physiological-anxiety-escalation.hf.space`, FastAPI, with `/ingest`, `/predict/{user_id}`, `/history/{user_id}`, `/feedback/anxiety`.

Work required is small:

1. Accept `subject_id` alongside `user_id` on `/ingest` and `/predict`.
2. After computing a forecast in `get_escalation_forecast`, POST a `Contribution` to the fusion service. Score = peak of the scaled 10-step forecast ÷ 100. `score_semantics = "peak_escalation_risk_10min_uncalibrated"`.
3. **Fix the in-memory normalisation store.** `set_norm_params` currently caches per-user baselines in process memory. Every Space restart silently wipes every participant's calibration and the forecast quietly changes meaning. Move it to the Postgres from §4 or to Influx.
4. Return `status = "insufficient_data"` when a user has no norm params, rather

than a forecast computed against defaults.

6.2 C2 — Digital phenotyping (Senu)

The GLOBEM validation documented in digital_phenotyping_page.dart — AUROC 0.548 [0.516–0.588] against a permutation null of 0.487, cross-cohort transfer 0.478 — does not support a risk score.

Deploy the service anyway, with status = "not_validated". It computes and returns the behavioural observation payload the app already renders (z-scores against personal baseline, data-quality flags, blocking issues), and it posts a contribution whose status prevents any composite from using it.

This is the right call for a demo: the modality is visible, the pipeline is exercised end-to-end, and no claim is made that the evidence cannot carry. If a reviewer asks why the behavioural panel shows no risk number, the answer is one sentence and it is a strong one.

Endpoint: POST /v1/observe → { observations[], baseline_ready, blocking_issues[] }.

6.3 C3 — Contextual / intervention (Uvindu)

Exists as c3_api/, FastAPI, artifacts committed. To containerise: wrap the existing app/ with §5's Dockerfile, keep /v3/* routes for backward compatibility, add /v1/contribute-side posting.

Four client-side schema violations to fix first — C3Gateway.classify() in tcwpn_mobile_app/lib/data/api/gateways.dart will 422 on every call today:

Problem	Fix
composite_risk is Field(...) (required) and is never sent	Either compute it server-side from the three modality risks, or send it. Server-side is better — the formula should live in one place.
Gateway sends null for physiological_risk / behavioral_risk / textual_risk / last_reward_norm; schema types are non-optional float	Change schema to Optional[float] = None and impute server-side, and return which fields were imputed in the response.
models.dart:128 maps unknown gender to 3; schema is ge=1.0, le=2.0	Widen the schema or map unknown to a defined value. Do not silently coerce.
risk_tier_enc is force-zeroed by a	Correct — leave it. Document why in

Problem	Fix
validator	the paper.

6.4 C4 — TC-WPN (Dulhara)

This service does not exist yet. tcwpn_test contains training and evaluation code only — no FastAPI, no Dockerfile, no app.py, and no checkpoint committed. The clinician app's README points at dulharakaushalya-tc-wpn-demo.hf.space; nothing in any repo builds it.

Before writing the handler, resolve one question that determines what the service *is*:

> **Where does the support set come from at inference time?**
>
> - **MIMIC-IV support notes** → the service is a meta-trained classifier applied to NHSL text. That is cross-corpus transfer, and you have no evidence for it.
> - **NHSL support notes, supplied by the clinician** → the service is genuine few-shot clinical adaptation, which is the claim in the proposal, and the support_set field the clinician app already sends is the right mechanism.
>
> Pick the second. Then the SupportSetScreen in the clinician app is not a convenience feature, it is the method.

Handler sketch:

```
# app/inference.py  (C4)
import torch, torch.nn.functional as F
from transformers import AutoTokenizer
from tcwpn.model import build_model        # vendored from src/tcwpn/

_MODEL, _TOK = None, None
MODEL_VERSION = "tcwpn-aux-k5-seed42@2026-08-14"

def _load():
  global _MODEL, _TOK
  if _MODEL is None:
    ckpt = hf_hub_download(REPO, "tcwpn_full_k5_seed42.pt", token=HF_TOKEN)
    _MODEL = build_model({"preset": "tcwpn_full", "projection_dim": 256})
    _MODEL.load_state_dict(torch.load(ckpt, map_location="cpu")["model"])
    _MODEL.eval()
    _TOK = AutoTokenizer.from_pretrained("emilyalsentzer/Bio_ClinicalBERT")
  return _MODEL, _TOK
```

```python
@torch.no_grad()
def predict(note_text, note_date, support_set):
    model, tok = _load()
    # support_set: [{text, label, note_date}], label in {0,1}
    sup_emb, sup_days, sup_lab = embed(support_set, note_date, tok, model)
    protos = []
    for c in (0, 1):
        m = sup_lab == c
        p, w = model.build_prototype(sup_emb[m], sup_days[m])
        protos.append(p)
    q = embed_one(note_text, tok, model)
    logits = model.classify(q, protos)
    p_anx = F.softmax(logits, dim=-1)[0, 1].item()
    return {
        "p_anxiety": p_anx,
        "score_semantics": "p_anxiety_uncalibrated",   # see §10.3
        "model_version": MODEL_VERSION,
        "support_n": len(support_set),
    }
```

Three deployment notes specific to this component:

- **Cold start is the worst of the five.** Bio_ClinicalBERT is ~440 MB and CPU inference on 2 cores is slow. Call `_load()` in a FastAPI `lifespan` hook so the cost is paid on Space wake, not on the clinician's first request. The clinician app's `TcwpnGateway.health()` already exists for exactly this — call it on app launch.
- **torch CPU wheel only.** Put `--extra-index-url https://download.pytorch.org/whl/cpu` in `requirements.txt` or the image pulls ~2.5 GB of CUDA you cannot use.
- **`return_attention` / `return_support_contributions`** are requested by the client. `build_prototype` already returns per-support weights `w`. Return them — they are the only genuinely interpretable output this model has. Do **not** fabricate token-level attention spans; return the field absent and let the UI degrade, which `gateways.dart` already handles.

6.5 Fusion (Uvindu)

Not a model. A registry. Routes:

```
POST /v1/subjects           (clinician token)  → create subject + pairing code
POST /v1/subjects/pair      (client token)     → redeem pairing code
POST /v1/contribute         (service token)    → store a Contribution
GET  /v1/state/{subject_id}   (client token)    → SubjectState
GET  /v1/history/{subject_id} (client token)    → readings over a window
GET  /health                                   → liveness + which deps are up
```

Contribute handler:

```python
@router.post("/v1/contribute", response_model=SubjectState)
async def contribute(c: Contribution, _=Depends(service_auth), db=Depends(get_db)):
    if c.contract_version.split(".")[0] != CONTRACT_VERSION.split(".")[0]:
        raise HTTPException(409, f"Contract major mismatch: {c.contract_version}")
    await db.execute(
        """INSERT INTO modality_readings
           (subject_id, component, score, score_semantics,
            status, captured_at, model_version, detail)
           VALUES ($1,$2,$3,$4,$5,$6,$7,$8)""",
        c.subject_id, c.component, c.score, c.score_semantics,
        c.status, c.captured_at, c.model_version, c.detail)
    await audit(db, actor=c.component, action="contribute", subject_id=c.subject_id)
    return await build_state(db, c.subject_id)
```

---

7. The composite score — the rule

SubjectState.composite is Optional[float] and defaults to None. It is populated **only** when all of the following hold, and composite_basis always says which condition failed:

1. At least two modalities report status == ok.
2. None of them is stale (captured_at within 72 h — matches Env.stalenessThreshold in the clinician app).
3. Every contributing modality's score_semantics appears in an explicit allow-list of quantities the team has agreed are combinable.
4. A weight vector exists for exactly that set of contributing modalities.

On condition 4, note that **three different weight vectors exist in the codebase right now**:

Source	C1	C2	C3	C4
R26-DS-012/ README.md	0.40	0.35	—	0.25
c3_api/ artifacts/ feature_cols.json	0.25	0.20	—	0.40
tcwpn_mobile_app/lib/core/ config/ env.example.dart	0.25	0.20	0.15	0.40

Pick one, delete the other two, and record where the numbers came from. If the answer is “we chose them” rather than “we fitted them”, say so in the paper — a stated prior is defensible, an unexplained one is not.

Do not renormalise silently over missing modalities. gateways.dart sends renormalise_on_missing: true. Renormalising a 4-modality weighting down to 1 reporting modality produces a number numerically equal to that modality alone but labelled “composite”, which is the most misleading possible output. Return the single modality with composite: null and composite_basis: "only c4_clinical_nlp reporting; composite requires ≥2".

Also delete Env.defaultWeights and the FusionResult.local(...) fallback path from the clinician app. A client-computed provisional composite is a second source of truth, and there is no version of this system where that ends well.

---

8. Standard operational practice

8.1 Health and readiness

```
@app.get("/health")
async def health():
    return {
        "status": "ok",
        "component": COMPONENT,
        "model_loaded": _MODEL is not None,
        "model_version": MODEL_VERSION,
        "contract_version": CONTRACT_VERSION,
```

```
    "fusion_reachable": await ping_fusion(),
    "commit": os.getenv("SPACE_COMMIT", "unknown"),
  }
```

8.2 Keeping Spaces awake for the demo

Free Spaces sleep on inactivity. Add a GitHub Actions cron in R26-DS-012 (you already have .github/workflows/weekly-physiological-pipeline.yml, so the pattern is established):

```
name: keepalive
on:
  schedule: [{ cron: "*/30 * * * *" }]
  workflow_dispatch:
jobs:
  ping:
    runs-on: ubuntu-latest
    steps:

     - run: |

        for u in "$C1" "$C2" "$C3" "$C4" "$FUSION"; do
          curl -sf --max-time 120 "$u/health" >/dev/null && echo "ok $u" || echo "DOWN $u"
        done
       env:
        C1: ${{ vars.C1_URL }}
        C2: ${{ vars.C2_URL }}
        C3: ${{ vars.C3_URL }}
        C4: ${{ vars.C4_URL }}
        FUSION: ${{ vars.FUSION_URL }}
```

On demo day, run workflow_dispatch 20 minutes before, and warm C4 by hand.

8.3 Timeouts, retries, idempotency

- Client → Space: 180 s for inference (cold start), 25 s for reads.
- Space → fusion: 10 s, 3 retries with exponential backoff, **fire-and-forget**.

  A failed contribution must never fail the caller's own response.
- Contributions carry (subject_id, component, captured_at, model_version).

  Make that combination the idempotency key so retries don't duplicate rows.

8.4 CORS

allow_origins=["*"] in c3_api/app/main.py is fine for a Flutter mobile client (no browser origin) but must not survive into any web dashboard. Set it to the explicit dashboard origin now.

8.5 Logging

Structured JSON, one line per request: timestamp, component, route, subject_id, status, latency_ms, model_version. **Never log note text, raw sensor values, or MRNs.** Space logs are visible to anyone with write access to the Space.

8.6 If any real participant data touches this

Free HF Spaces give no uptime guarantee and no BAA. If the demo uses real consented NHSL patients rather than synthetic ones:

- No raw note text at rest anywhere. C4 scores in memory and discards.
- MRN_PEPPER hashing per §1.2, applied client-side.
- Written data-flow approval from Dr. Suraweera before the first real note.
- The apps must say, on screen, that this is research decision support and not a

  diagnostic device.

If the demo uses synthetic patients, none of §8.6 applies — and for a first public demo, synthetic is the right choice.

8.7 Versioning

Bump MODEL_VERSION on every checkpoint change. It rides on every contribution and lands in modality_readings.model_version, so any historical state can be attributed to the exact model that produced it.

---

9. Bring-up order and smoke tests

Do these in order. Do not skip ahead; each step's failure mode is unambiguous only if the previous step passed.

Step	Action	Pass criterion
1	Rotate C3_SECRET_KEY, purge .env from git history	Secret no longer in any commit
2	Provision Postgres, run §4 DDL	\dt shows 5 tables
3	Deploy fusion Space, no other service	GET /health 200, db_reachable: true
4	POST /v1/subjects then /v1/subjects/pair with curl	One row in subjects, two in subject_aliases
5	POST /v1/contribute with a hand-written C4 payload	GET /v1/state/{id} shows 1 reporting, 3 missing, composite: null
6	Deploy C4, call /predict with a demo note + 5-note support set	Returns p_anxiety in (0,1), latency logged
7	Wire C4 → fusion	State shows a real C4 reading
8	Repeat 6–7 for C1, C2, C3	Four modalities in state
9	Point clinician app Env.* at the three URLs, rebuild	Chart renders four modality tiles
10	Point patient app at fusion, run pairing	Same subject_id on both devices
11	Two subjects, disjoint data	Neither state contains the other's readings — **this is the per-patient separation test, run it explicitly**

Step 11 is the one that most often silently fails. Write it as an automated test, not a manual check.

---

10. Constraints the services must respect

Engineering cannot fix these; it can only avoid overstating them.

10.1 C2 has no validated risk score

GLOBEM: AUROC 0.548 [0.516–0.588], permutation null 0.487, cross-cohort transfer 0.478, decision-curve net benefit ≈ 0.6 cases per 1,000. status: "not_validated" is the correct representation. Do not let it into a composite.

10.2 C3 is not a fusion model

feature_cols.json records composite_risk = (0.25·physiological + 0.20·behavioral + 0.40·textual) / 0.85, feeds that as feature F9, and targets risk_tier — a tiering of the same quantity. SHAP in the same file: physiological_risk 2.832, composite_risk 1.085, **textual_risk 0.079**. It was trained on NHANES (n=2,705) and Colombian survey data (n=2,657), not on anyone in this study. Present it as the intervention recommender it is. The fusion *registry* is a separate service and must not be described as a learned fusion model, because there is no cohort with all four modalities and a label on which to fit or validate one.

10.3 TC-WPN's named mechanisms are not what produces its score

Phase 4, five seeds, paired against aux_only (0.7371):

config	AUROC	Δ	paired t p	seeds better	dz
temporal_aux	0.7377	+0.0006	0.906	2/5	0.056
pcw_aux	0.7291	−0.0080	0.150	1/5	−0.797
tcwpn_full	0.7377	+0.0006	0.886	1/5	0.068

Neither w^T nor w^C shows a detectable effect once the auxiliary CE head is held constant. Every aux-free arm collapsed (proto_cos ≥ 0.9997, p_sd ≤ 0.0018), so the Stage C headline ΔAUROC of 0.239 is measuring the auxiliary head against a degenerate baseline, not the temporal or consistency mechanism.

Consequences for the deployed service:

- score_semantics = "p_anxiety_uncalibrated". There is no calibrator in the

repo; ECE ≈ 0.085, Brier ≈ 0.209. The clinician app reads TcwpnResult.calibratedProbability and .confidence — **neither exists**. Either fit a calibrator on a held-out split and say so, or rename the client fields to match what the model actually returns.

- The measured 0.738 [0.718–0.758] is *concurrent detection* on MIMIC-IV at prevalence 0.594. It is not a prospective claim and not an NHSL claim.
- The UI must not attribute the score to recency weighting. gateways.dart currently documents exp(−λΔt/365), λ = 0.5 as a fixed constant; in the current model λ is a learned log_lambda, and the mechanism it implements has no measured effect.

10.4 Documentation that contradicts the code

- R26-DS-012/README.md still gives TC-WPN's archived entropy formulation w_confidence = 1/(1 + β·H(x)). The implemented weight is w^C = exp(β·max(cos(z_i, p̃_c), 0)).
- tcwpn_mobile_app/README.md describes a shell/portal_launcher.dart + lib/tcwpn/ + lib/c3/ layout that no longer exists in lib/.

Fix both before the demo. A supervisor reading the README and then the code will find these in under five minutes.

Appendix — per-owner checklist

Owner	Component	Tasks
Dewdu	C1	Accept subject_id; move norm params off in-process memory; POST contributions; insufficient_data when uncalibrated
Senu	C2	Containerise observation pipeline; status: not_validated; POST contributions

Owner	Component	Tasks
Uvindu	C3	Containerise c3_api; fix 4 schema violations (§6.3); POST contributions
Uvindu	Fusion	Provision Postgres; build registry (§4–§7); own the tokens
Dulhara	C4	**Build the service from scratch**; resolve support-set question; push checkpoint to a model repo; CPU torch wheel; lifespan preload
Team	—	Rotate leaked key; agree one weight vector; fix both READMEs; write the step-11 isolation test

The three things that will decide whether this works, ahead of the mechanics:

Patient separation is currently broken at the identity layer, not the API layer. The patient app keys on user_id from SharedPreferences; the clinician app keys on patientMrn typed by the doctor. Those are two namespaces. Push C1 under "P007" and C4 under "NHSL-2026-114" and the fusion service holds two unrelated records that never join. Section 1 has the pairing-code design that fixes it — one canonical subject_id, app identifiers demoted to aliases, MRN hashed with a server-side pepper before it leaves the clinic.

Your fusion node should be a registry, not a model. Each modality pushes its latest reading; the fusion service stores and serves a per-patient panel. The alternative — fusion synchronously calling four sleeping Spaces — means four sequential cold starts on a client that already allows 180s for one. It also doesn't match the cadences: C1 fires every 60s, C2 daily, C4 only when a clinician writes a note. And the clinician app already implements POST /contribute + GET /state/{mrn}, so you'd be building against a client that expects push.

Your own component is the one that doesn't exist yet. C1 is live, C3 has a working FastAPI with committed artifacts, C2 has a pipeline. tcwpn_test has training and evaluation code and no serving layer, no Dockerfile, no committed

checkpoint. That's the longest pole, and §6.4 has the handler sketch plus the three CPU-deployment traps (torch CUDA wheel, lifespan preload, writable HF cache).

One question in §6.4 you should answer before writing a line of it: where does the support set come from at inference? MIMIC notes make this cross-corpus transfer with no evidence behind it. NHSL notes supplied by the clinician make it the few-shot adaptation your proposal claims — and then the `SupportSetScreen` already in the app isn't a convenience feature, it's the method.